

Chapter 3: Applying Evolution to Vehicle Routing Problems

Nature-Inspired Optimisation for Delivery Problems (2022)

Lecture Notes

2026

Chapter Outline

- 1 3.1 Evolutionary Computation
- 2 3.2 Evolutionary Algorithms
- 3 3.3 Applying Evolution to the CVRP
- 4 3.4 Choosing Parameters
- 5 3.5 Implementation
- 6 3.6 Results
- 7 3.7 Conclusions

3.1 Evolutionary Computation — The Big Idea I

Where does the idea come from? Darwin's *On the Origin of Species* (1859) proposed that species improve over generations through natural selection: individuals that are better adapted to their environment are more likely to survive and reproduce, passing their favourable traits to offspring. Evolutionary Computation borrows this mechanism and applies it to mathematical optimisation.

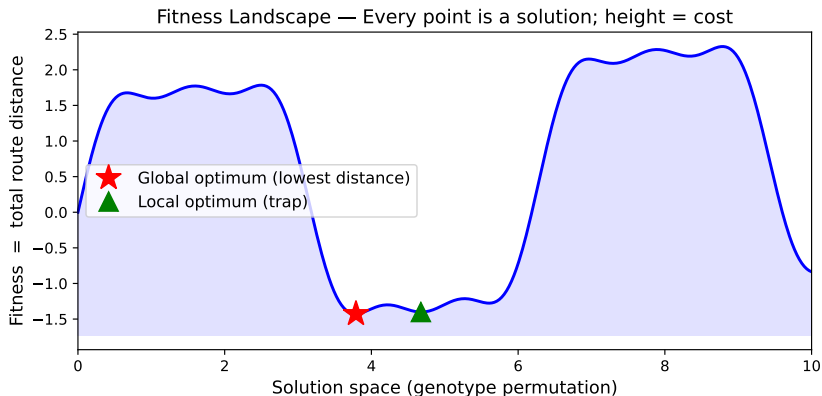
3.1 Evolutionary Computation — The Big Idea II

Core analogy

Biology	Optimisation
Individual organism	A candidate solution
Genome (DNA sequence)	Encoded representation of a solution
Fitness (survival rate)	Objective value (e.g. total distance)
Population	Set of candidate solutions
Reproduction	Creating new solutions
Mutation	Small random changes to a solution
Crossover	Combining parts of two solutions
Natural selection	Preferring better solutions

3.1 Evolutionary Computation — The Big Idea III

The fitness landscape — a mental model. Imagine every possible solution laid out on a map. The height of the map at any point represents the fitness (cost) of that solution. The goal is to find the lowest valley on this landscape.



3.1 Evolutionary Computation — The Big Idea IV

The landscape is almost always *multi-modal*: there are many local valleys (local optima) and a single deepest valley (the global optimum). A simple local search will fall into whichever valley it first encounters. An evolutionary algorithm maintains a *population* of solutions spread across the landscape, allowing it to explore many valleys simultaneously and escape local optima through crossover and mutation.

3.1 Evolutionary Computation — The Big Idea V

Why is this especially relevant to the CVRP? The Capacitated Vehicle Routing Problem (CVRP) is NP-hard: exact methods become computationally infeasible for large instances. The search space grows factorially with the number of customers, so exhaustive enumeration is impossible. Evolutionary algorithms offer a practical alternative: they do not guarantee the global optimum, but they consistently find good solutions in reasonable time, and their flexibility means they can handle constraints such as vehicle capacity naturally through the solution representation.

Key Insight

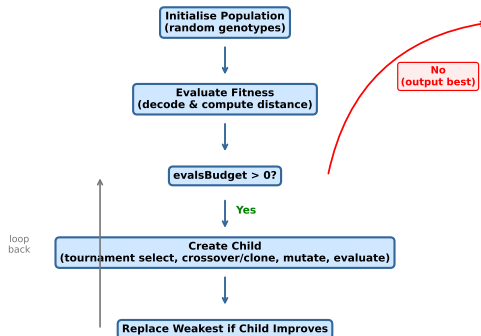
Evolutionary Algorithms do not require the objective function to be smooth, differentiable, or even well-defined mathematically — they only need to *evaluate* a solution. This makes them extremely versatile for combinatorial problems like VRP.

3.2 Evolutionary Algorithms — Structure and Components I

An Evolutionary Algorithm (EA) maintains a **population** of candidate solutions and improves them over many *generations* (iterations) by applying biologically inspired operators. The book describes the specific variant used by the ACE Doughnuts case study, which follows the standard EA cycle shown below.

3.2 Evolutionary Algorithms — Structure and Components II

The Evolutionary Algorithm Cycle



Each cycle generates one new child solution, evaluates it, and uses it to replace one of the weaker individuals in the population. This continues until a fixed *evaluation budget* is exhausted.

3.2 Evolutionary Algorithms — Structure and Components III

Algorithm 8: The Evolutionary Cycle (formal statement)

Procedure EvolutionaryAlgorithm()

- ➊ Initialise a population of N (N = population size) random solutions.
- ➋ Evaluate the fitness of every individual (compute total route distance).
- ➌ **While** the evaluation budget has not been exhausted:
 - ➊ Create a *child* solution (see child creation procedure below).
 - ➋ Select a *weak* individual from the population via tournament selection.
 - ➌ **If** the child is strictly better than the weak individual, replace the weak individual with the child.
 - ➍ Track the best solution seen so far.
- ➍ **Return** the best solution found.

Note that the replacement step is *conditional*: the child only enters the population if it improves on the individual it would replace. This ensures the average quality of the population never degrades, while still allowing new genetic material to enter if it is beneficial.

3.2 Evolutionary Algorithms — Structure and Components IV

The five components of an EA in detail:

(1) Representation. Before any evolution can happen, we must choose how to *encode* a solution as a string of genes (the *genotype*). For the CVRP, the book uses a *grand-tour* encoding: the genotype is a single permutation of all customer IDs. The *phenotype* (the actual vehicle routes) is *decoded* from the genotype on demand by greedily splitting the tour wherever the vehicle capacity would be exceeded. We discuss this in detail in Section 3.3.

(2) Initialisation. The initial population is created by generating N random permutations of the customer list. Diversity is important here: a population of near-identical solutions will converge quickly to a local optimum, whereas a diverse population explores the search space more thoroughly.

(3) Fitness evaluation. Each individual is scored by decoding its genotype into routes and summing the total travel distance. Lower distance means higher fitness (the problem is minimisation).

3.2 Evolutionary Algorithms — Structure and Components V

(4) Selection — Tournament Selection. To choose a parent for reproduction, the algorithm uses *tournament selection*: draw k (k = tournament size, usually 2) individuals at random from the population, and select the one with the *lowest* fitness value (shortest distance).

3.2 Evolutionary Algorithms — Structure and Components VI

Tournament Selection (size=2): draw 2 at random, keep the one with lower fitness

Population pool

Ind A
fit=450

Ind D
fit=290

Ind B
fit=380

Ind E
fit=420

Ind C
fit=510

Ind F
fit=360

Tourn. 1

Tourn. 2

Selected parents

WINNER:
Ind B (fit=380)

WINNER:
Ind D (fit=290)

3.2 Evolutionary Algorithms — Structure and Components VII

Tournament selection provides *selection pressure*: better individuals are more likely to be chosen, but there is always a small chance that a worse individual is selected, which maintains diversity and prevents premature convergence. The parameter k controls the pressure: larger k means only the very best individuals are selected, reducing diversity; smaller k means more randomness.

3.2 Evolutionary Algorithms — Structure and Components VIII

(5) Replacement — Reverse Tournament Selection. To decide *which* member of the population to potentially replace, the algorithm runs a tournament in reverse: draw k individuals at random and select the one with the *highest* fitness (worst solution). If the new child is better than this worst individual, it replaces it; otherwise, the population is unchanged. This strategy removes weak individuals while preserving good ones.

Key Insight: Steady-State vs. Generational EA

The book's EA is *steady-state*: it replaces one individual per iteration. A *generational* EA would replace the entire population each generation. Steady-state EAs update the population more smoothly and tend to preserve diversity better on a limited evaluation budget.

3.3 Applying Evolution to the CVRP — Representation I

The core challenge: how do we encode a VRP solution as a chromosome?

A VRP solution is a *set of routes*, each starting and ending at the depot. Naively encoding routes directly creates problems for crossover: combining two valid route sets can produce duplicated or missing customers. The book solves this by using the *grand-tour* representation.

Grand-Tour Representation

The **genotype** is a single permutation of all n customer IDs, in the order they would be visited on a hypothetical single long tour. The **phenotype** (actual routes) is *decoded* from the genotype by scanning through the list and starting a new route each time adding the next customer would cause the total demand to exceed the vehicle capacity Q (Q = capacity limit).

3.3 Applying Evolution to the CVRP — Representation II

CVRP Chromosome: Genotype (permutation) decoded into Phenotype (routes)

Genotype: single flat permutation of all customer IDs (the 'grand tour')



Phenotype: decoded into separate vehicle routes (each route starts and ends at depot D)



3.3 Applying Evolution to the CVRP — Representation III

Worked example of decoding. Suppose capacity $Q = 20$ and the genotype is $[3, 7, 1, 9, 4, 2, 8, 5, 6]$ with demands d_i (d subscript i = demand of customer i):
 $d_3 = 8, d_7 = 7, d_1 = 5, d_9 = 9, d_4 = 6, d_2 = 7, d_8 = 6, d_5 = 8, d_6 = 7.$

3.3 Applying Evolution to the CVRP — Representation IV

Worked Example: Decoding the Genotype

Step	Customer	Demand	Route load so far	Action
1	3	8	8	add to Route 1
2	7	7	15	add to Route 1
3	1	5	20	add to Route 1 (exactly at limit)
4	9	9	9	$20+9 > 20$, start Route 2
5	4	6	15	add to Route 2
6	2	7	22	$15+7 > 20$, start Route 3
7	8	6	6	add to Route 3
8	5	8	14	add to Route 3
9	6	7	21	$14+7 > 20$, start Route 4

Result: Route 1 = D→3→7→1→D, Route 2 = D→9→4→D, Route 3 = D→2→8→5→D, Route 4 = D→6→D.

3.3 Applying Evolution to the CVRP — Representation V

The decoding always produces a *feasible* solution (capacity never violated), because every genotype — even a random one — decodes to valid routes. This is an important property: the EA *never* wastes evaluations on infeasible solutions.

3.3 Applying Evolution to the CVRP — Representation VI

Genetic Operators — Crossover.

Crossover (also called *recombination*) combines two parent genotypes to produce a child. For permutations, a simple two-point crossover would create duplicates, so the book uses *Order-1 Crossover (OX)*:

Order-1 Crossover Procedure

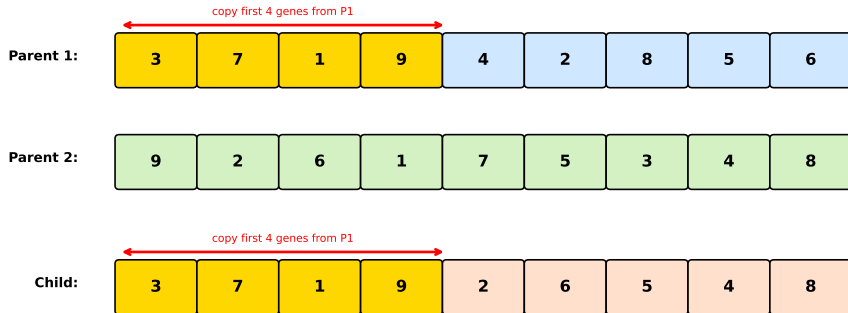
Given two parent permutations P1 and P2:

- 1 Choose a random crossover point x (a position in the chromosome).
- 2 The child inherits genes $0, 1, \dots, x - 1$ directly from P1 (preserving order and position).
- 3 The remaining genes are filled in from P2, scanning P2 left-to-right and adding each gene that has not yet appeared in the child.

The result is a valid permutation containing every customer exactly once.

3.3 Applying Evolution to the CVRP — Representation VII

Order-1 Crossover: copying a segment from P1, then filling from P2



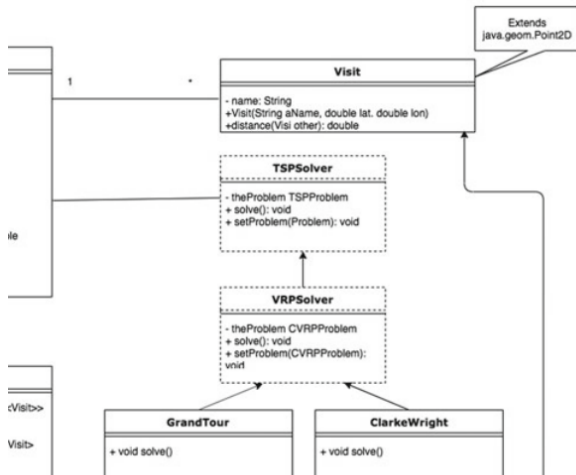
Gold cells copied directly from P1. Remaining cells filled in the order they appear in P2.

3.3 Applying Evolution to the CVRP — Representation VIII

Why does crossover help? If P1 visits customers $\{3, 7, 1\}$ in a good sub-sequence (because they are geographically close), and P2 has a good sub-sequence for $\{9, 4, 2, 8\}$, the child may combine both good segments. This is the *building block hypothesis*: short sequences of genes that work well together (called *building blocks*) tend to be preserved and combined across generations.

3.3 Applying Evolution to the CVRP — Representation IX

33



3.4 Choosing Parameters — Tuning the EA I

Evolutionary algorithms have several hyperparameters whose values significantly affect performance. The book discusses the following parameters and their recommended values for the CVRP application.

Algorithm 9: Child Creation — Parameter Summary

Parameter	Value used	Role
Population size	1,000	Controls diversity of the search
Evaluation budget	1,000,000	Total number of fitness evaluations
Crossover rate	0.5	Probability of applying crossover
Mutation rate	1.0	Always apply mutation (to every child)
Selection pressure	tournament size 2	Strength of parent selection
Replacement pressure	tournament size 2	Strength of replacement selection

The *crossover rate* of 0.5 means that there is a 50% chance the child is created by combining two parents; otherwise, the child is a copy (*clone*) of a single parent. In both cases, mutation is always applied. A

3.4 Choosing Parameters — Tuning the EA II

mutation rate of 1.0 means every child undergoes mutation — this is intentionally high to maintain diversity throughout the run.

3.4 Choosing Parameters — Tuning the EA III

Child creation procedure in detail (Algorithm 9).

Algorithm 9: CreateChild()

- ➊ With probability equal to the *crossover rate*:
 - ➊ Select *parent1* via tournament selection.
 - ➋ Select *parent2* via tournament selection.
 - ➌ Create the child via *crossover(parent1, parent2)*.
- ➋ Otherwise (no crossover):
 - ➊ Select a single *parent* via tournament selection.
 - ➋ The child is a *copy* of that parent.
- ➌ Apply *mutation* to the child.
- ➍ *Evaluate* the child (decode genotype, compute total distance).
- ➎ **Return** child.

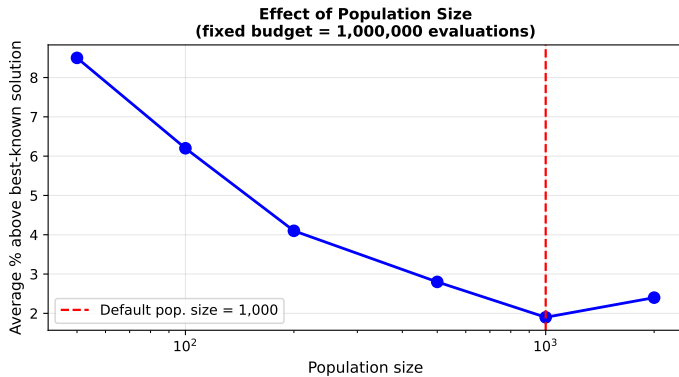
This two-path design allows the EA to exploit building blocks via crossover when it is productive, while still exploring via cloning-and-mutation when crossover would break good sub-sequences.

3.4 Choosing Parameters — Tuning the EA IV

Why is parameter choice hard?

Different problem instances may respond differently to the same parameter settings. A large population helps with diversity but means fewer generations are completed within a fixed evaluation budget. A high crossover rate can disrupt well-evolved sub-sequences; a very low rate loses the benefit of combining building blocks.

3.4 Choosing Parameters — Tuning the EA V



3.4 Choosing Parameters — Tuning the EA VI

The figure above illustrates a common pattern: solution quality improves as population size increases up to a point, then levels off or even worsens because each individual receives fewer evaluations within the budget. In practice, practitioners run the algorithm multiple times with different settings (parameter tuning) and select the configuration that performs best on average. The book cites Eiben and Smit (2012) as a reference on parameter tuning methods.

Key Insight

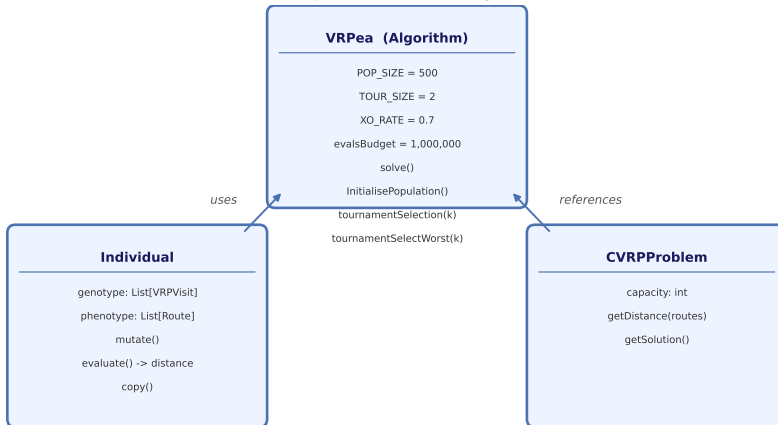
There is no universally optimal parameter setting. The parameters recommended in this chapter (pop. size = 1,000; mutation rate = 1; crossover rate = 0.5) were chosen empirically for the Augerat CVRP benchmark instances and provide a reasonable starting point for similar problem sizes.

3.5 Implementation — Architecture Overview I

The book implements the EA in Java (the *VRPea* class). This section explains the design, which can be straightforwardly translated to Python.

3.5 Implementation — Architecture Overview II

VRPea Implementation — Key Classes



The architecture has three main components:

3.5 Implementation — Architecture Overview III

- **VRPea** — the algorithm class, which contains the evolutionary cycle (Algorithm 8) and all selection/replacement logic.
- **Individual** — represents one candidate solution, holding both the genotype (the permutation) and the decoded phenotype (the routes). It also contains the `mutate()` and `evaluate()` methods.
- **CVRPPProblem** — holds the problem instance data (customer demands, coordinates, vehicle capacity) and computes the total distance of a solution.

The separation between `Individual` and `CVRPPProblem` is clean: the individual owns the solution representation; the problem owns the data. This design makes it easy to swap out the problem type or the representation without rewriting the evolutionary loop.

3.5 Implementation — Architecture Overview IV

The Individual class — genotype and phenotype.

The genotype is stored as an `ArrayList<VRPVisit>` (a list of customer visit objects). The phenotype is stored as an `ArrayList<ArrayList<VRPVisit>`» (a list of routes, each route being a list of visits). Crucially, the phenotype is **lazy**: it is set to `null` initially and only computed when `evaluate()` is called. If the genotype is later mutated, the phenotype is invalidated (set back to `null`) so it will be recomputed on the next evaluation.

3.5 Java: The Main Evolutionary Loop (Listing 3.1)

The following Java code is the core of the `VRPea.solve()` method. It implements Algorithm 8 exactly as described above.

```
private RandomSingleton rnd = RandomSingleton.getInstance();
private int POP_SIZE = 500;
private int TOUR_SIZE = 2;
private double XO_RATE = 0.7;
private int evalsBudget = 1000000;

@Override
public void solve() {
    // Reference to the best individual in the population
    Individual bestSoFar = InitialisePopution();
    while (evalsBudget > 0) {
        // Create child
        Individual child = null;
        if (rnd.getRnd().nextDouble() < XO_RATE) {
            // Create a new Individual using recombination,
            // randomly selecting the parents
            child = new Individual(super.theProblem,
                                   tournamentSelection(TOUR_SIZE),
                                   tournamentSelection(TOUR_SIZE));
        } else {
            // Create a child by copying a single parent
            child = tournamentSelection(TOUR_SIZE).copy();
        }
        child.mutate();
    }
}
```

3.5 Java: Genotype/Phenotype and Constructors (Listings 3.2 & 3.3)

```
// The genotype is a "grand tour" list of visits
private ArrayList<VRPVisit> genotype;
// The phenotype is a set of routes created from the genotype
private ArrayList<ArrayList<VRPVisit>> phenotype;
```

Listing 2: Listing 3.2 – Genotype and phenotype fields within Individual

```
public Individual(CVRPPProblem prob) {
    // Constructor to create a new random genotype
    problem = prob;
    genotype = new ArrayList<VRPVisit>();
    for (Visit v : prob.getSolution()) {
        genotype.add((VRPVisit)v);
    }
    genotype = randomize(genotype);
    phenotype = null;
}

public Individual(CVRPPProblem prob, Individual parent1, Individual parent2) {
    // Create a new Individual via recombination of parent1 and parent2
    problem = prob;
    genotype = new ArrayList<VRPVisit>();
    int xPoint = rnd.getRnd().nextInt(parent1.genotype.size());
    for (int count = 0; count < xPoint; count++) {
        genotype.add(parent1.genotype.get(count)); // copy p1[0..xPoint)
    }
    for (int count = 0; count < parent2.genotype.size(); count++) {
```

3.5 Java: Mutation and Evaluate (Listings 3.4 & 3.5)

Mutation (Listing 3.4) randomly removes one gene and re-inserts it at a new position. Notice that the phenotype is immediately invalidated (set to null) after mutation, so the outdated route set is never accidentally used.

```
public void mutate() {  
    // Mutate the genotype by randomly moving a gene.  
    phenotype = null;  
    int rndGene = rnd.getRnd().nextInt(genotype.size());  
    VRPVisit v = genotype.remove(rndGene);  
    int addPoint = rnd.getRnd().nextInt(genotype.size());  
    genotype.add(addPoint, v);  
}
```

Listing 4: Listing 3.4 – Individual.mutate()

Evaluate (Listing 3.5) lazily builds the phenotype, then calls the problem's `getDistance()` method. It scans the genotype left-to-right; each time the next customer would cause the route's total demand to exceed the capacity, a new route is started.

```
public double evaluate() {  
    if (phenotype == null) {  
        phenotype = new ArrayList<ArrayList<VRPVisit>>();  
        ArrayList<VRPVisit> newRoute = new ArrayList<VRPVisit>();  
        for (VRPVisit v : genotype) {  
            if (v.getDemand() + routeDemand(newRoute) > problem.getCapacity()) {  
                phenotype.add(newRoute);  
                // capacity exceeded: start new route  
                newRoute = new ArrayList<VRPVisit>();  
            }  
            newRoute.add(v);  
        }  
        phenotype.add(newRoute);  
    }  
    return problem.getDistance(phenotype);  
}
```

3.5 Python Translation — Core EA Loop

The following Python code faithfully translates the Java EA into idiomatic Python. Understanding the Python version is useful for experimentation, as it is more accessible than the Java original.

```
import random
import copy

class VRPea:
    def __init__(self, problem, pop_size=500, tour_size=2,
                  xo_rate=0.7, evals_budget=1_000_000):
        self.problem = problem      # CVRPPProblem instance
        self.POP_SIZE = pop_size    # number of individuals in population
        self.TOUR_SIZE = tour_size  # tournament size for selection
        self.XO_RATE = xo_rate      # crossover probability
        self.evals_budget = evals_budget # total evaluation count

    def solve(self):
        population = self._initialise_population()
        best = min(population, key=lambda ind: ind.evaluate())

        while self.evals_budget > 0:
            if random.random() < self.XO_RATE:
                # Crossover: select two parents, combine them
                p1 = self._tournament_select(population)
                p2 = self._tournament_select(population)
                child = Individual(self.problem, p1, p2)
            else:
                # Clone: copy one parent
```

3.5 Python Translation — Individual Class

```
class Individual:
    def __init__(self, problem, parent1=None, parent2=None):
        self.problem = problem
        self.phenotype = None    # lazily computed
        self.fitness = None

        if parent1 is None:
            # Random initialisation
            self.genotype = list(problem.customers)
            random.shuffle(self.genotype)
        else:
            # Order-1 crossover of two parents
            xp = random.randint(0, len(parent1.genotype))
            self.genotype = list(parent1.genotype[:xp])
            for gene in parent2.genotype:
                if gene not in self.genotype:
                    self.genotype.append(gene)

    def mutate(self):
        """Remove one random gene and re-insert it at a new position."""
        self.phenotype = None    # invalidate cached phenotype
        self.fitness = None
        idx = random.randint(0, len(self.genotype) - 1)
        gene = self.genotype.pop(idx)
        new_pos = random.randint(0, len(self.genotype))
        self.genotype.insert(new_pos, gene)
```

3.5 Python Translation — Selection and Hybrid Solver

```
def _tournament_select(self, population):
    """Return the individual with the lowest fitness from a random sample."""
    sample = random.sample(population, self.TOUR_SIZE)
    return min(sample, key=lambda ind: ind.evaluate())

def _tournament_select_worst(self, population):
    """Return the individual with the highest fitness from a random sample."""
    sample = random.sample(population, self.TOUR_SIZE)
    return max(sample, key=lambda ind: ind.evaluate())

def _initialise_population(self):
    """Create POP_SIZE random individuals and evaluate them all."""
    pop = [Individual(self.problem) for _ in range(self.POP_SIZE)]
    for ind in pop:
        ind.evaluate()
    return pop

# -- Hybrid solver: run Clarke-Wright first, then refine with EA --
def hybrid_solve(problem, n_ea_runs=20):
    """
    Run Clarke-Wright to get a warm start, then run the EA 20 times
    (because the EA is stochastic) and report the best result found.
    """
    cw = ClarkeWright(problem)
    cw_solution = cw.solve()
    cw_distance = problem.total_distance(cw_solution)
```


3.6 Results — Benchmarking on Augerat Instances I

ACE Doughnuts test the EA on the **Augerat (1995) benchmark** instances — a well-known collection of CVRP test problems from the literature. The instances are grouped into three sets: **Set A** (31–80 customers), **Set B** (31–78 customers), and **Set P** (16–101 customers). For each instance, the optimal (or best-known) solution from exact branch-and-cut methods is available, allowing a direct comparison.

Because the EA is stochastic, it is run **20 times** per instance. The reported result is the *best* solution found across the 20 runs.

Interpretation Caveat

The EA rarely finds the provably optimal solution (that would require an exact solver), but it consistently finds solutions close to optimal in far less time. The question is how large the gap is.

3.6 Results — Benchmarking on Augerat Instances II

Results on Set A (Table 3.1 excerpt).

Problem	n	B&C best	EA distance	Vehicles	% over B&C
A-n32-k5	31	784	797.9	5	1.77%
A-n33-k5	32	661	662.1	5	0.17%
A-n33-k6	32	742	742.7	6	0.09%
A-n34-k5	33	778	786.4	5	1.08%
A-n36-k5	35	799	810.1	5	1.39%
A-n37-k5	36	669	682.3	5	1.99%
A-n38-k5	37	730	736.5	5	0.90%
A-n39-k5	38	822	845.1	5	2.81%
A-n44-k7	43	937	978.6	6	4.44%
A-n45-k6	44	944	963.7	6	2.05%

The symbol n denotes the number of customers (excluding the depot). “B&C best” is the best solution found by branch-and-cut (Augerat 1995). “% over B&C” is the percentage by which the EA’s best result exceeds the B&C result. On smaller instances (e.g. A-n33-k5, A-n33-k6), the EA comes within 0.1–0.2% of the optimal solution.

3.6 Results — Benchmarking on Augerat Instances III

Results on Set B (Table 3.2 excerpt).

Problem	n	B&C best	EA distance	Vehicles	% over B&C
B-n31-k5	31	672	676.1	5	0.61%
B-n34-k5	34	788	790.9	5	0.36%
B-n38-k6	38	805	809.7	6	0.58%
B-n39-k5	39	549	564.7	5	2.86%
B-n41-k6	41	829	839.7	6	1.29%
B-n43-k6	43	742	758.1	6	2.17%
B-n44-k7	44	909	947.3	7	4.22%
B-n50-k8	50	1313	1342.0	8	2.21%
B-n52-k7	52	747	778.2	7	4.17%
B-n57-k9	57	1598	1668.6	9	4.42%

Set B results are similar in character to Set A: on smaller instances the gap is very small, while on larger, harder instances (more customers, more vehicles) the gap grows somewhat. The EA *always* finds the same number of vehicles as the B&C solution, indicating that the route structure is correct even when the exact distances differ slightly.

3.6 Results — Benchmarking on Augerat Instances IV

Results on Set P (Table 3.3 excerpt) — includes larger instances.

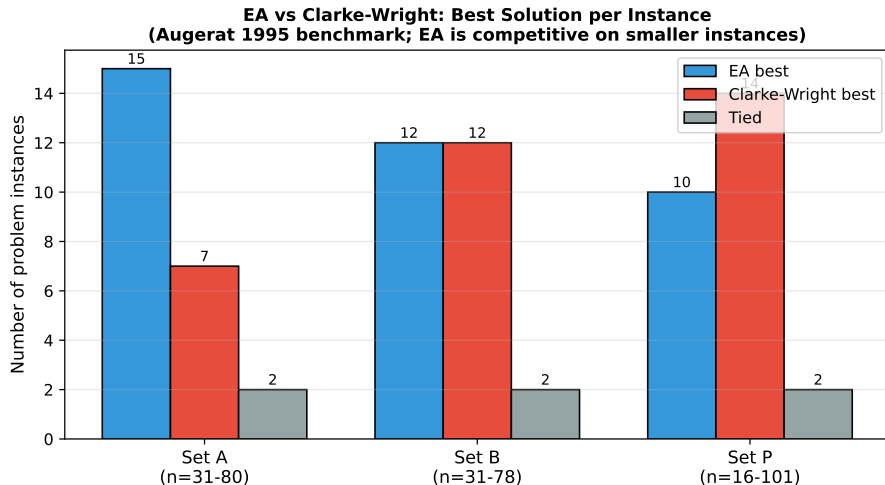
Problem	n	B&C best	EA distance	Vehicles	% over B&C
P-n16-k8	16	435	451.3	8	3.76%
P-n19-k2	19	212	212.7	2	0.31%
P-n20-k2	20	220	217.4	2	−1.17%
P-n22-k2	22	216	217.9	2	0.86%
P-n40-k5	40	458	464.7	5	1.46%
P-n45-k5	45	510	535.7	5	5.04%
P-n55-k7	55	524	586.2	7	11.87%
P-n60-k10	60	856	996.4	16	16.40%
P-n70-k10	70	834	942.4	10	13.00%
P-n101-k4	101	681	929.9	4	36.56%

Three remarkable observations: (1) On P-n20-k2, the EA actually finds a *shorter* route than the branch-and-cut result (−1.17%), likely due to rounding differences between software platforms. (2) On very large instances (P-n101-k4, 101 customers), the gap widens considerably (36.56%). (3) On some large instances, the EA uses more vehicles than the B&C solution.

3.6 Results — Benchmarking on Augerat Instances V

EA versus Clarke-Wright: who finds the best solution? (Table 3.4 summary)

3.6 Results — Benchmarking on Augerat Instances VI



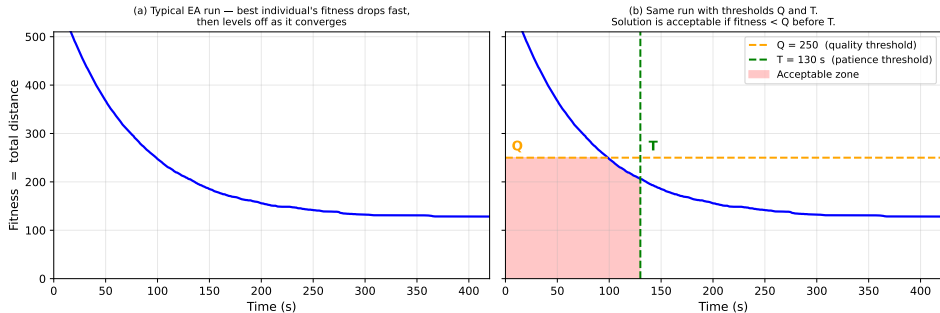
3.6 Results — Benchmarking on Augerat Instances VII

The figure above summarises the Table 3.4 comparison. The EA tends to win on smaller instances (Set A), where population-based search can effectively explore the space within the budget. Clarke-Wright becomes more competitive on larger instances (Set P), which is not surprising: Clarke-Wright is a deterministic greedy heuristic that scales well, while the EA's population diversity advantage diminishes when each individual represents a complex large tour.

3.6 Results — Benchmarking on Augerat Instances VIII

The convergence behaviour of the EA.

EA Convergence (Fig. 3.3 — reproduced)



3.6 Results — Benchmarking on Augerat Instances IX

The left plot shows a typical EA run: the fitness (total distance) drops rapidly in the first few minutes as the population quickly improves, then slows down as the algorithm approaches a local optimum. This is the characteristic *convergence* behaviour of EAs.

The right plot introduces two user-defined thresholds: Q (Q = quality threshold) is the maximum distance the user will accept, and T (T = patience threshold, in seconds) is the maximum time the user is willing to wait. A solution is considered *acceptable* if the fitness drops below Q before time T . The shaded region is the “acceptable zone”: if the convergence curve enters this zone, the algorithm has found a satisfactory solution in time.

3.7 Conclusions I

What has been achieved in this chapter?

The chapter demonstrated how a biologically-inspired Evolutionary Algorithm can be applied to the Capacitated Vehicle Routing Problem. Starting from the analogy of natural selection, the chapter built a complete, working EA for the CVRP, covering representation, operators, selection, and parameter choice.

Summary of the EA approach

- The *grand-tour genotype* encodes any CVRP solution as a simple permutation of customer IDs, guaranteeing feasibility after decoding.
- *Order-1 crossover* combines building blocks from two parents while preserving the constraint that each customer appears exactly once.
- *Random relocation mutation* keeps the search exploring and prevents the population from converging prematurely.
- *Tournament selection* (for both parents and replacement) provides tunable selection pressure without requiring fitness scaling.

3.7 Conclusions II

Key findings from the benchmarks.

- On smaller instances (31–50 customers), the EA finds solutions within 1–3% of the branch-and-cut optimum, which is excellent for a metaheuristic.
- The EA consistently uses the same number of vehicles as the optimal solution, meaning it finds the correct routing structure even if individual routes are not perfectly optimised.
- On larger instances (60–101 customers), the gap grows — the EA still finds good solutions, but they are further from optimal.
- Neither the EA nor Clarke-Wright dominates the other across all instances: the best strategy depends on the specific instance size and structure.

Practical recommendation for ACE Doughnuts: Run both Clarke-Wright and the EA (multiple times), then take the best result. The book's Algorithm 3.6 (the hybrid solver) does exactly this. This portfolio approach hedges against the weaknesses of each individual method.

3.7 Conclusions III

Limitations and open questions.

- ❶ **Stochasticity** — Because the EA is random, it can produce a different result each run. Running it 20 times mitigates this but adds computational cost. A real deployment would need to balance solution quality against available time.
- ❷ **Parameter sensitivity** — The parameters (population size, crossover rate, etc.) were tuned for the Augerat instances. They may need re-tuning for significantly different problem sizes or structures.
- ❸ **Large instance scalability** — The 36% gap on P-n101 shows that the basic EA does not scale as well as heuristics for very large instances. More sophisticated operators (e.g. local search within the EA, also known as a *memetic algorithm*) could close this gap.
- ❹ **Single objective** — This chapter optimised total distance only. Real-world delivery problems often have multiple objectives (time windows, driver costs, CO₂ emissions), which are addressed in later chapters.

3.7 Conclusions IV

Chapter Takeaway

Evolutionary Algorithms offer a flexible, powerful approach to the CVRP. They require no problem-specific mathematical analysis, scale to real-world instance sizes, and can be adapted to new constraints by modifying only the representation and evaluation function. Their main cost is the need for many fitness evaluations and careful parameter selection.

3.7 Conclusions V

References cited in this chapter.

- Augerat, P. (1995). *Approche polyédrale du problème de tournées de véhicules*. PhD Thesis, Grenoble Institute of Technology.
- Augerat, P. (2014a/b/c). VRP-REP benchmark datasets (Sets A, B, P).
- Darwin, C. (1859). *On the Origin of Species by Means of Natural Selection*. John Murray, London.
- Eiben, A.E. & Smit, S.K. (2012). Evolutionary Algorithm Parameters and Methods to Tune Them. In *Autonomous Search*, Springer.
- Goldberg, D. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley.
- Holland, J.H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

Chapter 3 — One-Slide Summary

What is an Evolutionary Algorithm?

A population-based search method inspired by natural selection. It maintains N candidate solutions and iteratively creates children via crossover and mutation, replacing weak individuals when children improve on them.

CVRP Representation

Genotype: permutation of customer IDs (grand tour).

Phenotype: routes decoded by splitting at capacity limits.

Any permutation decodes to a *feasible* solution.

Bottom Line

EA finds solutions within 1–5% of optimal on medium instances in reasonable time. Combining with

Key Parameters

Population size	1,000
Evaluation budget	1,000,000
Crossover rate	0.5
Mutation rate	1.0 (always)
Tournament size	2

Operators

Selection: tournament (draw 2, keep best).

Crossover: Order-1 (copy prefix from P1, fill from P2).

Mutation: remove one gene, re-insert at random position.

Replacement: reverse tournament (remove worst).